

07 - Leader Election

Nei sistemi distribuiti spesso è necessario che una singola entità coordini il lavoro delle altre entità per la risoluzione dei task. La presenza di tale entità può essere necessaria per *semplificare* lo svolgimento di un task oppure perché è richiesto dalla natura stessa del problema.

Elezione di un leader

Il problema della scelta di un **leader** tra un insieme omogeneo e simmetrico di individui di una popolazione è noto come il **Leader Election Problem**. Generalmente il problema richiede di portare la rete da una configurazione iniziale in cui tutti i nodi sono in uno stesso stato, ad una configurazione in cui tutti i nodi saranno in uno stato *follower* eccetto uno solo che dovrà essere nello stato di *leader*. Non ci sono restrizioni su quali nodi devono iniziare il task della leader election, e nemmeno su quali nodi possono diventare leader.

Si può pensare alla leader election come un metodo che **forza** la restrizione di *iniziatore unico*, infatti eletto un leader è possibile risolvere il task con protocolli a *single initiator* semplicemente ponendo il leader come nodo iniziatore.

E' possibile definire formalmente il problema con una tripla $P = \langle P_{init}, P_{final}, R \rangle$, dove

- $P_{init} : \forall x \in V, \text{state}(x) = \text{asleep}$
- $P_{final} : \exists! x \in V : \text{state}(x) = \text{Leader} \wedge \forall y \neq x : \text{state}(y) = \text{Follower}$
- $R = \begin{cases} \text{Total Reliability (TR)} \\ \text{Bidirectional Link (BL)} \\ \text{Connectivity (CN)} \end{cases}$

Allora vale il seguente teorema.

Teorema

Sotto le restrizioni R non è possibile risolvere il problema della leader election in modo **deterministico**.

Questo risultato di *impossibilità* molto forte è dato dalla condizione di **perfetta simmetria** nella rete. Il fenomeno della **perfetta simmetria** occorre quando una rete ha una struttura totalmente simmetrica, ossia, ogni nodo non è in grado di distinguersi da un altro nodo. Dunque se tutti i nodi si trovano nello stesso stato interno σ ad un istante t , formalmente, $\forall x, y \in V : \sigma(x, t) = \sigma(y, t)$, da quel momento in poi si comporteranno in modo identico, dato che avranno la stessa visione del mondo esterno: non potendosi comportare in maniera differente, i nodi cambieranno stato alla stessa maniera, e quindi non è possibile arrivare in

una situazione in cui un nodo avrà uno stato diverso dagli altri (il leader).

Il problema della leader election è anche noto come **symmetry breaking**, perché si cerca di "rompere" tale simmetria tra i nodi. Il teorema ci dice però che per rompere la simmetria, le restrizioni R non sono sufficienti. In particolare, è necessario aggiungere la restrizione **Initial distinct value (o Unique Identifier)**, nella quale si assume che ogni nodo x è **univocamente identificato** da un valore $id(x)$, cioè che rispetta la seguente proprietà

$$\forall x, y \in V \quad id(x) \neq id(y)$$

Elezione in un albero

Si osserva che per sistemi distribuiti in cui la topologia è quella ad albero, il problema della leader election può essere facilmente risolto tramite la **saturazione**. Infatti è sufficiente usare la saturazione per calcolare il nodo x con $id(x)$ minimo. Tale nodo sarà eletto a leader, mentre i restanti diventeranno follower.

Elezione in un anello

Si considera ora un sistema distribuito con topologia ad **anello**. In particolare sia $R = (\{x_1, \dots, x_n\}, E)$ un anello con n nodi ed n archi in cui ogni nodo è in grado di distinguere la direzione: ogni nodo può scegliere di mandare un messaggio alla sua destra o alla sua sinistra. In particolare, l'insieme degli archi è

$$E = \{(x_i, x_{i+1}) : i = 1, 2, \dots, n-1\} \cup \{(x_n, x_1)\}$$

Si osserva che gli id non sono globalmente **consistenti**, ossia non è detto che $id(x_i) = i$ o che se $id(x_i) = k$ allora $id(x_{i+1}) = k+1$ e $id(x_{i-1}) = k-1$.

Si discutono tre protocolli diversi che risolvono il problema della leader election su questa topologia. In particolare questi protocolli permettono di individuare il nodo x con $id(x)$ minimo su una rete ad anello, in modo da poter eleggere tale x come leader.

Protocollo 1: All the way

L'idea del primo protocollo, detto **all the way**, consiste nell'inviare ogni etichetta ($id(\cdot)$) ad ogni nodo.

In particolare, quando un nodo x si attiva (spontaneamente o alla ricezione di un messaggio Election, si vedano i dettagli nel protocollo) inoltra un messaggio in direzione concorde a tutti (ad esempio destra) con all'interno $id(x)$. Quando un nodo riceve un messaggio Election con l' id di qualcun'altro, invia il proprio id se non lo ha ancora fatto, aggiorna la sua etichetta minima vista in precedenza con l'ultima ricevuta (se necessario) e inoltra in avanti il messaggio.

Il processo può essere visualizzato come segue: Ogni entità crea un messaggio contenente il proprio id , e questo messaggio attraversa tutta la rete.

Ogni entità, per via dei vincoli di total reliability e finite communication delay, vedrà entro un

tempo finito gli id di tutti gli agenti, incluso quello di valore minimo. Potrà quindi determinare se è il minimo o meno, e quindi, se entrare nello stato leader o follower.

Ci si chiede quando si verifica tale evento, ossia, quando un'entità termina la propria esecuzione.

Si osserva che le entità inoltrano solo messaggi contenenti valori diversi dal proprio. Quando un nodo x riceve un messaggio contenente $id(x)$, allora vuol dire che questo messaggio ha toccato tutti i nodi dell'anello, dunque x non lo inoltra nuovamente. Quindi, ciascun valore attraverserà tutta la rete una singola volta. Si osserva che nell'anello, ad un certo istante, circolano n messaggi diversi contenenti tutti gli identificativi dei nodi. La ricezione del proprio id da parte di un nodo, però, non è un evento sufficiente affinché questo possa terminare la sua computazione. Infatti, i messaggi non vengono ricevuti nello stesso ordine in cui questi sono stati inviati, e per tali ritardi di trasmissione, può verificarsi il caso in cui un agente riceva il proprio id prima di ricevere tutti i messaggi inviati dagli altri nodi. In qualche modo, ogni agente deve venir a conoscenza del numero n di nodi che costituiscono la rete affinché possa calcolare correttamente il minimo, e quindi terminare.

Tale valore non è noto a priori, essendo che la knowledge della ring size non è parte delle restrizioni del problema. Per conoscere n , ogni nodo x inserisce nel messaggio con il proprio id un contatore che viene incrementato ad ogni inoltro: il valore del contatore quando il messaggio torna a x sarà esattamente pari ad n . Allora quando x riceve il messaggio contenente $id(x)$, ricava il valore di n , essendo che tale messaggio deve aver attraversato l'intera rete, e si può comportare in due modi:

- se x ha ricevuto n id distinti (ogni x mantiene un contatore incrementato ogni volta che riceve un messaggio) allora calcola il minimo tra tutte le etichette per decidere se essere leader o follower;
 - se x ha ricevuto $k < n$ etichette allora deve attendere altre $n - k$ nuove etichette per poter calcolare il minimo.
- Allora, ogni nodo termina localmente, riuscendo ad identificare un leader globale e comune a tutti.

Formalmente il protocollo è il seguente. L'insieme degli stati è

- $S = \{\text{asleep}, \text{awake}, \text{follower}, \text{leader}\}$
 - $S_{init} = \{\text{asleep}\}$
 - $S_{term} = \{\text{follower}, \text{leader}\}$
- E valgono le restrizioni $R \cup \{\text{ID}, \text{Ring}\}$

Essendo che $\forall x \in V, |N(x)| = 2$, quando x riceve un messaggio da un suo vicino sender, si pone $\text{other} = N(x) \setminus \{\text{sender}\}$ per indicare il suo altro vicino

```
if state == "ASLEEP":
    spontaneously:
        INITIALIZE
        state = "AWAKE"
```

```

    Receiving("Election", value*, counter*):
        INITIALIZE
        send("Election", value*, counter*+1) to other
        min = MIN{min,value*}
        count = count +1
        state = "AWAKE"

if state == "AWAKE":
    receiving("Election", value*, counter*)
    if value != id(x):
        send("Election", value*, counter*+1) to other
        min = MIN{min, value*}
        count = count+1
        if known:
            CHECK
    else:
        ringsize = counter*
        known = True
        CHECK

Procedure INITIALIZE:
    count = 0
    size = 1
    known = False
    send("Election", id(x), size) to right
    min = id(x)

Procedure CHECK:
    if count == ringsize:
        if min == id(x):
            state = "LEADER"
        else:
            state = "FOLLOWER"

```

Si osserva che il protocollo appena descritto funziona anche per collegamenti *unidirezionali* e risolve il problema più generale della *data collection*.

Note

Si distinguono due contatori:

il contatore di un nodo, che incrementa ogni volta che riceve un messaggio (ciascun nodo riceve tutti gli id degli altri nodi una sola volta).

Il contatore allegato al messaggio, che incrementa ogni volta che attraversa un nuovo agente.

Correttezza e complessità

Il protocollo è corretto, in quanto l'assunzione di Total Reliability garantisce che ogni nodo prima o poi riceverà le informazioni riguardo le etichette di tutti quanti, dunque riuscirà a calcolare il minimo.

Il protocollo ha una message complexity

$$M(\text{all-the-way}) = O(n^2)$$

in quanto su ciascuno degli n archi passano n messaggi distinti, essendo che ogni entità genera un messaggio che attraverserà l'intero anello una singola volta.

Si può calcolare anche una **bit complexity** del protocollo. Si devono contare il numero di bit necessari per rappresentare $id(x)$ e il contatore. Se il messaggio $id+count$ si codifica in binario con $\log_2(id(x) + n)$, allora la bit complexity è pari a

$$M_{bit}(\text{all-the-way}) = O(n^2 \log(id(x) + n))$$

Ideal time complexity

Si assuma che il sistema sia sincrono, che tutti i nodi inizino il protocollo ad uno stesso istante t e che il tempo di trasmissione è di una sola unità fissata $\Delta = 1$; allora il tempo ideale è n .

Il caso peggiore è quando si attiva spontaneamente un solo nodo x_1 al tempo t_0 . Inviando il messaggio al suo vicino x_2 , esso si attiverà al tempo $t_1 = t_0 + 1$. Il nodo x_3 si attiverà al tempo successivo, e così via fino al nodo x_n che si attiverà dopo $n - 1$ istanti. Da quel momento in poi, l'ultimo nodo che terminerà il processo sarà proprio x_n , il quale dovrà aspettare che la sua etichetta faccia il giro completo dell'anello, risultando così in una time complexity di

$$T(\text{all-the-way}) = O(n)$$

Protocollo 2: As far as it can

Per definire un protocollo più efficiente, si osserva che nel protocollo *All the way* vi è una propagazione di messaggi superflui. Infatti, in tale protocollo, ogni messaggio attraversa tutto l'anello, ma tale propagazione dei messaggi non è necessaria: se un agente x avente $id(x)$ riceve un $id(y)$ di un agente y , e gli identificativi sono tali per cui $id(x) < id(y)$, allora x sa che $id(y)$ non sarà il più piccolo valore. Essendo che si vuole identificare l' id minimo tra tutte le entità, non è necessario che $id(y)$ continui a viaggiare nella rete.

Si modifica quindi il protocollo *All the way* affinché ogni nodo x blocchi l'inoltro di tutti i messaggi che contengono un id dal valore più grande di quello che x momentaneamente ritiene minimo. Allora un messaggio con un certo id viaggia in avanti più a lungo che può (**as far as it can**) e non in ogni caso (all the way).

Quindi, in questo protocollo, ogni entità genera un messaggio che attraversa l'anello sino a quando questo viene ricevuto o da un agente avente un id minore di quello contenuto nel messaggio, o dal suo mittente.

In merito alla terminazione, si osserva che certamente il nodo con etichetta minima sarà l'unico a ricevere un messaggio con la propria etichetta, questo perché i messaggi contenenti gli altri id non saranno in grado di tornare dai propri mittenti, visto che incontreranno nel loro percorso un agente contenente un id value minore. Perciò, il nodo avente id minimo saprà che tale valore è il più piccolo, e potrà quindi terminare *localmente* il suo compito, diventando *leader*.

In particolar modo, si osserva che, essendo che il messaggio contenente l' id minimo ha dovuto attraversare l'intera rete, allora ogni agente è a conoscenza del valore id contenuto in esso, e tale messaggio è l'unico che tornerà al suo mittente, ma gli altri agenti non possono sapere che questo sia effettivamente il minimo, ossia non sanno che non vi sono altri id aventi valore minore che circolano nella rete.

Il nodo con id minimo saprà quindi che è il leader, sia questo nodo l . Per far terminare anche gli altri nodi, il nodo l dovrà notificare (in broadcast) gli altri nodi che lui è il leader.

Si osserva che il costo aggiuntivo in bit complexity non è eccessivo, in quanto tutti i nodi certamente convergeranno su una singola etichetta minima, e quindi basterà un solo bit per terminare il task.

Formalmente il protocollo è il seguente. L'insieme degli stati è

- $S = \{\text{asleep, awake, follower, leader}\}$
 - $S_{init} = \{\text{asleep}\}$
 - $S_{term} = \{\text{follower, leader}\}$
- E valgono le restrizioni $R \cup \{\text{ID, Ring}\}$

Essendo che $\forall x \in V, |N(x)| = 2$, quando x riceve un messaggio da un suo vicino sender, si pone $\text{other} = N(x) \setminus \{\text{sender}\}$ per indicare il suo altro vicino

```
if state == "ASLEEP":
    spontaneously:
        INITIALIZE
        state = "AWAKE"

    Receiving("Election", value):
        INITIALIZE
        if value < min:
            send("Election", value) to other
            min = value
        state = "AWAKE"

if state == "AWAKE":
    receiving("Election", value):
        if value < min:
            send("Election", value) to other
            min = value
        else:
            if value == min:
```

NOTIFY

```
receiving(Notify):  
    send (Notify) to other  
    state = "FOLLOWER"
```

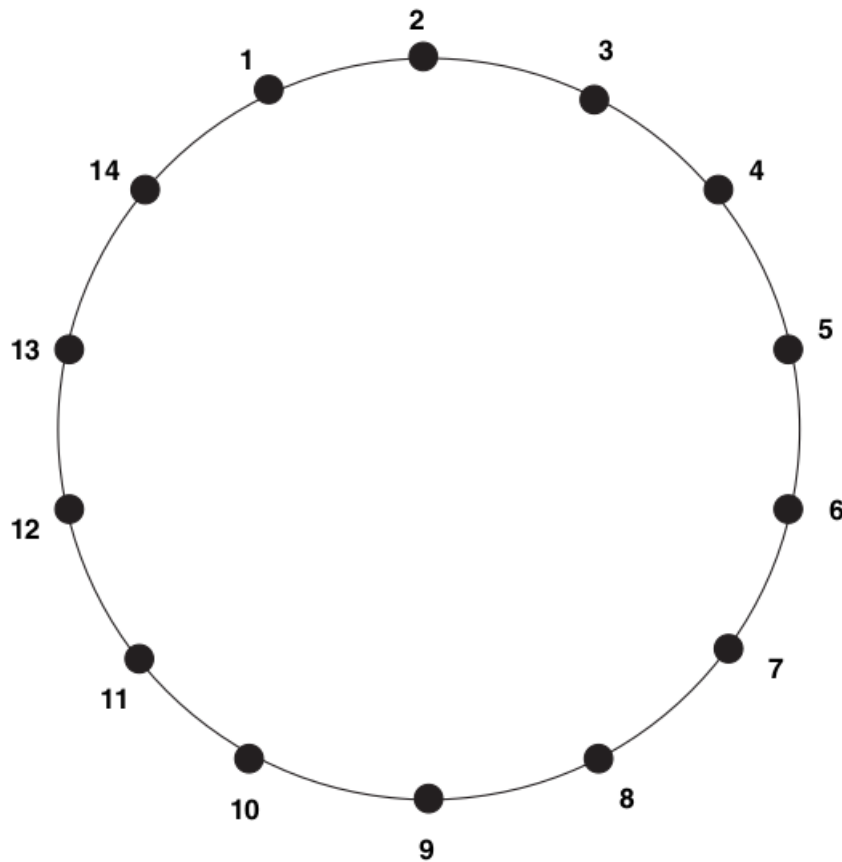
```
Procedure INITIALIZE:  
    send("Election", id(x)) to right  
    min = id(x)
```

```
Procedure NOTIFY:  
    send(Notify) to right  
    state = "LEADER"
```

Complessità

L'esatto numero di messaggi scambiati durante l'esecuzione del protocollo dipende da diversi fattori. Ad esempio, si consideri il costo causato dal messaggio Election originato dal nodo x . Tale messaggio viaggerà attraverso l'anello sino a quando non viene ricevuto da un nodo che ha ricevuto in precedenza un id minore, oppure sino a quando non raggiunge il proprio mittente. Quindi, il costo del suo viaggio dipende da come gli id sono allocati nella rete.

Si studia quindi il worst case: siano $id_1, id_2, \dots, id_n$ gli id dei vari agenti, ordinati in maniera crescente ($id_1 < id_2 < \dots < id_n$). L' id_1 attraverserà sempre tutto l'anello, avendo quindi un costo di n messaggi. L' id_2 verrà fermato solo dall'agente con id_1 . Quindi il suo costo nel worst case è di $n - 1$, ottenibile solamente se l'agente con id_2 è posizionato immediatamente dopo l'agente con id_1 nella direzione in cui viaggiano i messaggi. In generale, l' id_{i+1} verrà fermato da un qualsiasi agente avente id_j con $1 \leq j \leq i$, e avrà quindi un costo di $n - i$ messaggi. Ciò si verifica se tutte le entità aventi id_j con $1 \leq j \leq i$ sono posizionate l'una dopo l'altra (ossia se esiste un cammino che tocca tutti e soli i nodi aventi tali id) e l'agente con id_{i+1} è posizionato immediatamente dopo di essi nella direzione in cui viaggiano i messaggi. Infatti, tutti i worst cases per ciascuno degli id sono ottenuti simultaneamente quando gli id sono posizionati in un ordine circolare "crescente", e tutti i messaggi sono inviati nella direzione "crescente".



In questa situazione critica, contando anche il broadcast finale, la message complexity è pari a

$$n + \sum_{i=1}^n i = n + \frac{n(n+1)}{2}$$

dunque

$$M(\text{as-far}) = O(n^2)$$

Il caso migliore invece si ottiene quando il nodo con etichetta minima inizia il protocollo per primo e la sua etichetta riesce a fare il giro completo della rete prima che le altre vengano inoltrate al proprio vicino. In particolare, il messaggio con etichetta minima viene inoltrato n volte, mentre ogni messaggio con etichetta maggiore viene inoltrato una sola volta. In questa situazione, contando il broadcast finale, si ottiene message complexity

$$n + (n-1) + n$$

Protocollo 3: Stage technique

Nel protocollo *as far as it can* e nel protocollo *all the way* gli id viaggiano in modo incontrollato lungo l'anello. In particolare, nel protocollo *as far as it can* viaggiano finché non

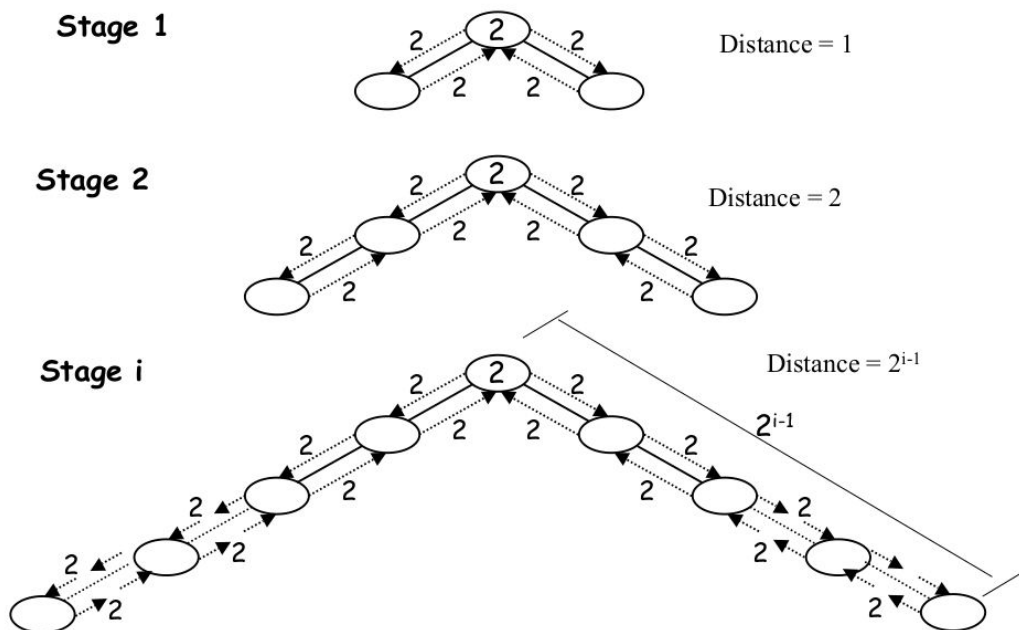
vengono filtrati da un nodo con id più basso, mentre nel protocollo *all the way* viaggiano lungo l'intero anello.

Un'idea è quella di **controllare** la trasmissione di un id ad una distanza **limitata** e di avere dei **feedback** per bloccarne o estenderne la trasmissione a distanze maggiori.

Quindi, in generale un agente può compiere più di un tentativo per diventare leader:

l'elezione avviene come una sequenza di "stage elettorali", dove all'inizio di ogni stage, i "candidati" partecipano all'elezione. Al termine di tale stage, alcuni "candidati" verranno sconfitti, non proseguendo il tentativo di diventare i leader, mentre gli altri inizieranno lo stage successivo. Si osserva esplicitamente che il concetto di stage è una nozione logica, e non richiede che il sistema sia sincronizzato. Infatti, delle componenti del sistema possono essere molto più veloci di altre, quindi entità distinte possono eseguire lo stesso stage in istanti di tempo totalmente diversi.

Un nodo attivo x esegue quindi il protocollo in **stage**, dove nello stage i il nodo x inoltra a destra e a sinistra la sua etichetta $id(x)$ fino a una distanza massima di 2^{i-1} salti.

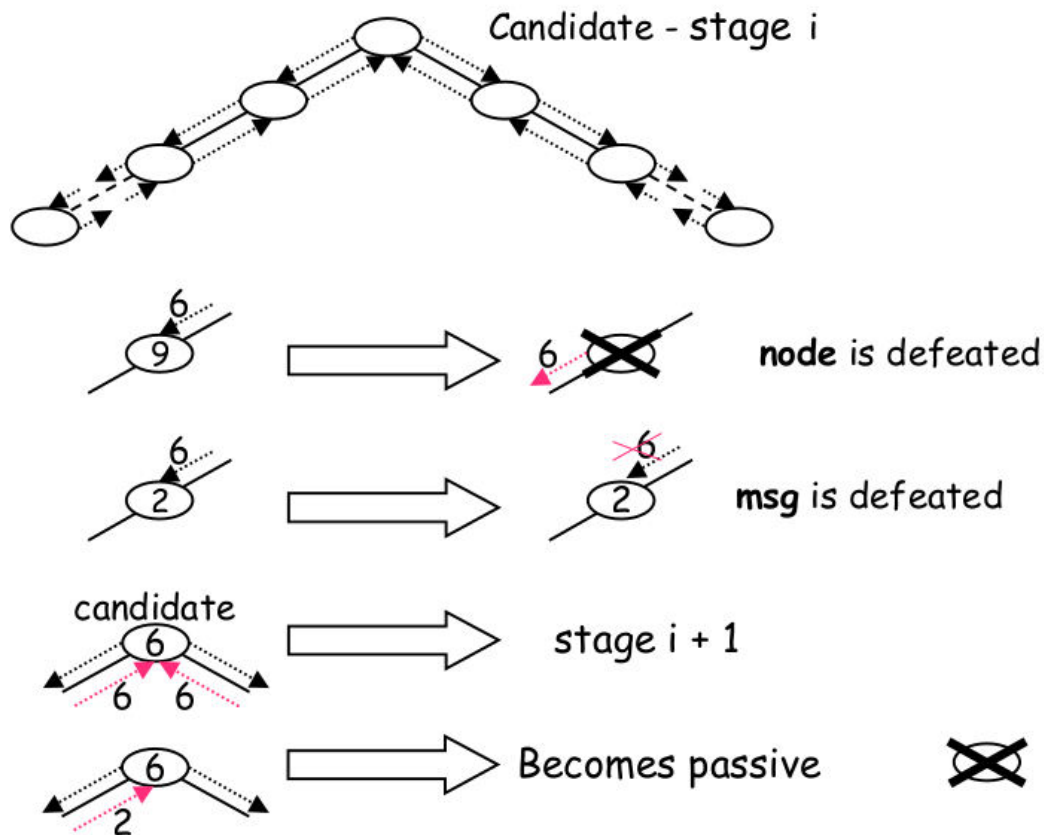


Quando un nodo y riceve $id(x)$ la inoltra al nodo successivo solo se $id(y) > id(x)$; in questo caso, certamente y non sarà mai eletto leader, dunque y passa in uno stato *defeated*, nel quale y potrà solamente inoltrare messaggi inviati da altri nodi.

Se $id(x)$ arriva a distanza 2^{i-1} allora verrà rispedito indietro al mittente con un messaggio di *feedback*. Se x riceve il feedback da entrambi i lati, allora tutti i nodi a distanza 2^{i-1} da lui hanno etichette maggiori, i quali verranno quindi sconfitti, dunque x deve inoltrare $id(x)$ a distanza maggiore. In tale caso, si dice che il nodo x è *sopravvissuto* allo stage i e parteciperà allo stage $i + 1$.

Se $id(x)$ non riceve il feedback da uno o entrambi i lati, allora esiste almeno un nodo z entro

2^{i-1} salti che ha etichetta minore $id(z) < id(x)$: allora ad un certo punto il nodo x passerà in stato defeated per colpa di $id(z)$.



Infine, quando un nodo x^* riceve il messaggio dal lato opposto in cui lo ha inviato, allora x^* potrà affermare di essere il leader. A questo punto x^* invia una notifica in broadcast a tutti gli altri nodi, che passeranno dallo stato defeated allo stato follower.

Riassumendo:

- in ogni stage i ci sono dei nodi candidati al ruolo di leader (i nodi sopravvissuti allo stage $i - 1$);
- ogni candidato invia il suo id da entrambi i lati dell'anello;
- il messaggio id viaggia finché: non incontra un nodo con id minore oppure non arriva a distanza 2^{i-1} ;
- se id arriva a distanza 2^{i-1} (ossia non incontra nessun id minore), allora torna indietro tramite messaggi di *feedback*;
- se un candidato riceve i messaggi di feedback da entrambi i lati, rimane candidato per lo stage $i + 1$.

Nel protocollo valgono le seguenti tre regole:

1. Se un nodo candidato riceve il suo id dal lato opposto a quello da cui l'ha mandato, diventa leader e lo notifica agli altri nodi;
2. Se un nodo candidato riceve un id minore del suo allora passa nello stato defeated;
3. Un nodo nello stato defeated può solamente inoltrare messaggi di altri nodi, e passerà nello stato follower quando riceve la notifica dal leader.

Si osserva nuovamente che durante l'esecuzione del protocollo è possibile che diversi nodi si trovino in diversi stages. Ci si chiede quindi cosa succede se un nodo viene "sconfitto" da un messaggio dello stage k , e immediatamente dopo tale evento riceve il suo messaggio di feedback dello stage $k - 1$.

E' sufficiente che i nodi rispettino l'ordine delle etichette e si comportino come descritto nel protocollo. Infatti, se in un qualsiasi istante un nodo riceve un id minore (proveniente da un qualsiasi stage), diventerà giustamente "sconfitto" e non attenderà più i suoi messaggi di feedback.

Correttezza e terminazione

Si osserva che in un tempo finito il protocollo porta sempre il sistema nello stato finale in cui si ha un leader e tutti follower. Infatti, dopo un tempo finito il nodo con etichetta minima si deve necessariamente svegliare. Tale nodo ad un certo stage i^* riceverà da un lato il messaggio che ha inviato dal lato opposto, inoltrato dai nodi sconfitti, dove

$$i^* = \min\{i \in \mathbb{N} : 2^{i-1} \geq n\} \iff i^* = O(\log_2 n)$$

Dunque il numero di stage necessari e sufficienti per far terminare il protocollo è $O(\log n)$. Il candidato che riceve il suo stesso messaggio dal verso opposto in cui lo ha inviato saprà che tutte le altre entità sono state sconfitte, e potrà quindi diventare leader. Dato che ogni stage dura per un tempo finito, ne segue che in un tempo finito viene eletto un leader che notifica tutti gli altri nodi rendendoli follower, facendoli terminare tutti correttamente.

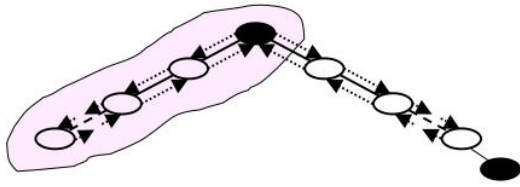
Complessità

Per quanto riguarda la message complexity, si contano il numero di messaggi che vengono inviati in ogni stage del protocollo. Si definisce la quantità

$$\begin{aligned} n_i &:= \text{numero di nodi che superano lo stage } i - 1 \\ &:= \text{numero di nodi che partecipano allo stage } i \end{aligned}$$

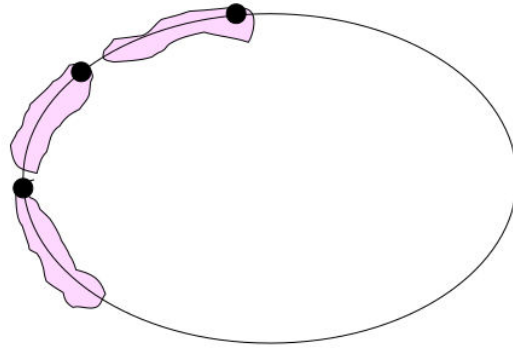
Si osserva che un nodo x supera lo stage $i - 1$ quando $id(x)$ è minore di $2^{i-2} id$ dei nodi alla sua destra e di $2^{i-2} id$ dei nodi alla sua sinistra. Dunque la fase $i - 1$ viene superata da al più un solo nodo ogni $2^{i-2} + 1$ nodi; quindi vale che

$$n_i \leq \frac{n}{2^{i-2} + 1}$$



So,

$$n_i \leq n / (2^{i-2} + 1)$$



Sia $i > 1$; si considerano i messaggi inviati nello stage i . Nel caso peggiore, la situazione è la seguente:

- **Messaggi "forth"**: i messaggi di tipo *forth* inviati nello stage i percorrono una distanza di 2^{i-1} salti in entrambe le direzioni. Dato che partecipano allo stage i esattamente n_i candidati, allora vengono inviati al più $n_i \cdot 2 \cdot 2^{i-1}$ messaggi di tipo *forth*;
- **Messaggi "feedback"**: Per ogni nodo che supera lo stage i vengono necessariamente trasmessi $2 \cdot 2^{i-1}$ messaggi *feedback*; ogni nodo che non supera la fase i invece potrà ricevere al più un messaggio *feedback*, per un totale di $(n_i - n_{i+1})2^{i-1}$ messaggi ulteriori. Complessivamente si trasmettono un numero di messaggi *feedback* pari a

$$n_{i+1} \cdot 2 \cdot 2^{i-1} + \underbrace{(n_i - n_{i+1})}_{\text{nodi che non superano lo stage } i} 2^{i-1}$$

Dunque il totale dei messaggi inviati per ogni stage $i > 1$ è al più

$$\begin{aligned} n_i \cdot 2 \cdot 2^{i-1} + n_{i+1} \cdot 2 \cdot 2^{i-1} + (n_i - n_{i+1}) \cdot 2^{i-1} &= 2^{i-1} \cdot (3 \cdot n_i + n_{i+1}) \\ &\leq 2^{i-1} \cdot \left(3 \cdot \left\lfloor \frac{n}{2^{i-2} + 1} \right\rfloor + \left\lfloor \frac{n}{2^{i-1} + 1} \right\rfloor \right) \\ &< \frac{3n \cdot 2^{i-1}}{2^{i-2} + 1} + \frac{n \cdot 2^{i-1}}{2^{i-1} + 1} \\ &< \frac{3n \cdot 2^{i-1}}{2^{i-2}} + \frac{n \cdot 2^{i-1}}{2^{i-1}} \\ &= 6n + n \\ &= 7n \end{aligned}$$

Per quanto riguarda il primo stage invece ($i = 1$), il caso peggiore si ha quando tutti i nodi sono iniziatori. In questo caso si ha che

$$n_2 \leq \frac{n}{2^0 + 1} = \frac{n}{2}$$

I nodi che superano il primo stage hanno un costo individuale di

$$\begin{cases} 2 \text{ per msg "forth"} \\ 2 \text{ per msg "feedback"} \end{cases} \implies 4 \cdot n_2 \text{ messaggi totali}$$

Gli altri invece, quelli che non lo superano, hanno un costo individuale di

$$\begin{cases} 2 \text{ per msg "forth"} \\ 1 \text{ per msg "feedback"} \end{cases} \implies 3 \cdot (n - n_2) \text{ messaggi totali}$$

In totale quindi vale

$$4n_2 + 3n - 3n_2 = n_2 + 3n < 4n$$

Combinando il tutto, si ottiene che il numero totale dei messaggi inviati è pari a

$$\begin{aligned} M(\text{STAGE TECHNIQUE}) &\leq \sum_{i=1}^{\log n} 7n + \overbrace{O(n)}^{\text{stage 1}} \\ &= 7n \cdot \log n + O(n) \\ &= O(n \log n) \end{aligned}$$

Elezione in una mesh

Si studia ora il problema dell'elezione di un leader all'interno di una mesh.

Una mesh M di dimensione $a \times b$ ha $n = a \cdot b$ nodi, siano questi $x_{i,j}$ con $1 \leq i \leq a$ e

$1 \leq j \leq b$. Ciascun nodo $x_{i,j}$ è connesso con $x_{i-1,j}, x_{i+1,j}, x_{i,j+1}, x_{i,j-1}$, se questi esistono. Si evidenzia che queste etichette $x_{i,j}$ sono solo a scopo descrittivo e non sono note alle entità.

Il numero totale di link è quindi $m = a(b-1) + b(a-1) = 2ab - a - b$.

In una mesh si distinguono tre tipi di nodi:

- I corners, o spigoli. Ve ne sono 4 e sono le entità aventi due vicini.
- I borders, ossia i nodi ai lati esterni della mesh. Hanno tre vicini e ve ne sono $2(a+b)$.
- I noti interni, aventi ciascuno di essi quattro vicini, e ve ne sono un totale di $n - 2(a+b-2)$

Mesh non orientate

L'asimmetria di una mesh può essere utilizzata al fine di sviluppare un protocollo ad hoc per l'elezione di un leader. Essendo che non importa quale entità diventi un leader, si può eleggere uno dei quattro spigoli. In questa maniera, il problema di eleggere un leader tra n possibili nodi si riduce al problema di scegliere un leader tra i 4 spigoli.

Si ricorda che un numero arbitrario di agenti può iniziare la computazione, ciascuno dei quali è ignaro del fatto che anche altri agenti possano aver iniziato l'esecuzione del protocollo e, nel caso in cui ciò accada, di quando e dove gli altri agenti inizieranno.

Quindi, per risolvere il problema, bisogna definire un protocollo che prima di tutto notifichi gli

spigoli del processo di elezione, per poi effettuare l'elezione tra di essi.

Questo primo passo può essere ottenuto facendo eseguendo un processo di wake-up di tutte le entità. Quando un'entità si sveglia (spontaneamente se è un initiator, o alla ricezione di un messaggio di wake-up da un altro agente altrimenti), le sue azioni successive dipenderanno dalla sua posizione all'interno di una mesh (bordo, spigolo o interno). Così facendo, i quattro spigoli si sveglieranno entro tempo finito, sempre per via della total reliability (e dell'assioma di finite transmission delay), e potranno iniziare il processo di elezione.

Si osserva che, se si considerano solo i bordi e gli spigoli della mesh, e i link tra essi, questi formano una topologia ad anello. Si può, quindi, eleggere un leader fra gli spigoli eseguendo un protocollo di elezione per gli anelli: gli spigoli saranno gli unici candidati, mentre i nodi sui bordi svolgeranno il ruolo di inoltrare i messaggi (saranno quindi dei defeated nodes). Quando uno spigolo viene eletto leader, notificherà tramite broadcasting tutti gli altri agenti affinché il protocollo termini. Riassumendo, il processo consisterà in:

1. Una fase di wake-up, iniziata dagli initiators;
2. Una fase di elezione, sull'anello esterno, tra gli spigoli.
3. Una fase di broadcasting iniziata dal leader.

Si analizzano individualmente le tre fasi:

1. Sia k il numero di initiators. Ciascuno di essi invierà un messaggio di wake up a tutti i suoi vicini. Un non initiator riceverà il messaggio di wake-up da un vicino e lo invierà a tutti i suoi altri vicini che, facendo riferimento ai tre tipi di nodi nella rete, possono essere al più tre. Di conseguenza, il numero di messaggi scambiati in questa fase è al più

$$3n + k$$

2. L'elezione sull'anello esterno richiede più attenzione. Prima di tutto bisogna scegliere quale protocollo per elezione su anelli utilizzare ed adattarlo al problema su mesh per assicurarsi che i messaggi di tale protocollo vengano instradati correttamente tra i link dell'anello esterno. Si prenda a titolo d'esempio il protocollo *Stages* visto in precedenza, e si consideri il primo stage. In accordo col protocollo, ogni candidato (nel nostro caso, un nodo su un angolo) invia un messaggio in entrambe le direzioni dell'anello, contenente il proprio valore. Ogni entità sconfitta (in questo caso, un nodo del bordo) inoltrerà il messaggio attorno l'anello. Quindi, nella mesh, ogni nodo su un angolo invierà un messaggio ai suoi soli due vicini. Si considerino ora i nodi sui bordi. Un nodo y sul bordo ha tre vicini, dei quali solamente due appartengono all'anello esterno. Quando y riceve il messaggio, non sa quale dei suoi altri due vicini appartiene all'anello, e, di conseguenza, non sa a quale port inoltrarlo. Per ovviare a tale problema, si inoltra il messaggio su entrambi i link. Il messaggio che viene inoltrato su un link che costituisce l'anello viaggerà su di esso, mentre l'altro raggiungerà un nodo interno z . Quando il nodo interno z riceve il messaggio un messaggio di elezione, essendo che ogni nodo conosce la sua tipologia (essendo questa dipendente dal

numero di link ad esso incidenti), risponderà al nodo di frontiera y con un messaggio che notifica il fatto che z è un nodo interno, in maniera tale che ulteriori messaggi di elezione non vengano più inoltrati ad esso. Nel protocollo *Stages*, il numero di candidati è almeno dimezzato ad ogni stage. Questo vuol dire che, dopo il secondo stage, uno tra i due nodi ai bordi che hanno superato lo stage precedente, determinerà se questo ha il più piccolo id tra i quattro candidati, diventando così leader. Ogni stage richiede $2n'$ messaggi, dove $n' = 2(a + b - 2)$ è la dimensione dell'anello esterno. Ulteriori $2(a + b - 4)$ messaggi vengono mandati, nel primo stage, dai nodi sui bordi ai nodi interni, essendo che questi ultimi devono ancora venir esclusi dal processo di inoltro dei messaggi di elezione. Nel conteggio dei messaggi, bisognerebbe prendere in considerazione anche le $2(a + b - 4)$ risposte provenienti da tali nodi interni, ma tali messaggi possono essere evitati. Quindi, il numero di messaggi per il processo di elezione sarà al più

$$4(a + b - 2) + 2(a + b - 4) = 6(a + b) - 16$$

Si osserva che in una mesh quadrata ($a = b$), questo processo di election ha un costo in $\mathcal{O}(\sqrt{n})$ messaggi.

3. Il broadcasting della notifica dell'elezione di un leader può essere fatto utilizzando *Flood*, che richiederà meno di $3n$ messaggi essendo che viene iniziato da un nodo su uno spigolo. Si può ulteriormente migliorare la fase di propagazione, assicurandosi che meno di $2n$ messaggi vengano mandati in tale fase.

Quindi, in totale, il protocollo di elezione su mesh appena definito avrà un costo di

$$6(a + b) + 5n + k - 16$$

dove k è il numero di iniziatori.

Si osserva esplicitamente che l'operazione più costosa è quella del wake up dei nodi.